Nama : Farhan Kurniawan

NIM : 2311104073

Kelas : SE-07-02

1. Link github: https://github.com/farkurr/KPL_FARHANKURNIAWAN_2311104073_SE-07-02.git

2. Membuat kelas KodeBuah.cs

```
JM4 | KodeBuah | kodeBuah

using System;
using System.Collections.Generic;

2 references
class KodeBuah
{
    private static Dictionary<string, string> kodeBuahMap = new Dictionary<string, string>
    {
        {"Apel", "A00"},
        {"Aprikot", "B00"},
        {"Alpukat", "C00"},
        {"Pisang", "D00"},
        {"Paprika", "E00"},
        {"Kurma", "K00"},
        {"Durian", "L00"},
        {"Anggur", "M00"},
        {"Melon", "N00"},
        {"Semangka", "O00"}
    };

    2 references
    public static string GetKodeBuah(string namaBuah)
    {
        return kodeBuahMap.ContainsKey(namaBuah) ? kodeBuahMap[namaBuah] : "Kode tidak ditemukan";
    }
}
```

Kode di atas adalah program dalam **C#** yang menggunakan **Dictionary** untuk menyimpan pasangan nama buah dan kode uniknya. Kelas KodeBuah memiliki Dictionary statis bernama kodeBuahMap, yang menyimpan daftar nama buah sebagai **key** dan kode unik sebagai **value**. Metode GetKodeBuah(string namaBuah) digunakan untuk mencari kode buah berdasarkan nama yang diberikan. Jika buah tersebut ada dalam Dictionary, metode akan mengembalikan kode yang sesuai, tetapi jika tidak ditemukan, akan mengembalikan string "Kode tidak ditemukan". Contoh penggunaannya dalam metode Main() adalah dengan memanggil KodeBuah.GetKodeBuah("Apel"), yang akan menghasilkan output "A00", sementara jika diberikan input yang tidak ada dalam daftar, seperti "Mangga", maka outputnya adalah "Kode tidak ditemukan". Program ini berguna untuk **mendapatkan kode buah berdasarkan namanya** dengan pencarian yang cepat dan efisien menggunakan **Dictionary**.

3. Membuat kelas PosisiKarakterGame.cs

```
JM4                                                    PosisiKarakterGame
1   using System;
2
3   class PosisiKarakterGame
4   {
5       private string state;
6
7       public PosisiKarakterGame()
8       {
9           this.state = "Berdiri";
10      }
11
        4 references
12      public void TekanTombol(string tombol)
13      {
14          switch (tombol)
15          {
16              case "W":
17                  Console.WriteLine("Tombol arah atas ditekan");
18                  break;
```

```
JM4                PosisiKarakterGame                TekanTombol(string tombol)
19              case "S":
20                  Console.WriteLine("Tombol arah bawah ditekan");
21                  break;
22              case "Berdiri":
23                  Console.WriteLine("Posisi standby");
24                  this.state = "Berdiri";
25                  break;
26              case "Tengkurap":
27                  Console.WriteLine("Posisi istirahat");
28                  this.state = "Tengkurap";
29                  break;
30              case "Terbang":
31                  Console.WriteLine("Posisi take off");
32                  this.state = "Terbang";
33                  break;
34              case "Jongkok":
35                  Console.WriteLine("Posisi landing");
36                  this.state = "Jongkok";
37                  break;
38              default:
39                  Console.WriteLine("Perintah tidak dikenali");
40                  break;
80 %   No issues found                                   Ln: 40   Ch: 23   SPC   CRI
```

```
40                  break;
41              }
42          }
43      }
44
```

Kode di atas adalah program dalam **C#** yang mengatur posisi karakter dalam permainan berdasarkan input tombol yang ditekan. Kelas PosisiKarakterGame memiliki atribut state untuk menyimpan status karakter, yang secara default diatur ke **"Berdiri"** melalui konstruktor. Metode TekanTombol(string tombol) menggunakan **switch-case** untuk menangani berbagai input: jika tombol "W" ditekan, program mencetak **"Tombol arah atas ditekan"**, sedangkan "S" mencetak **"Tombol arah bawah ditekan"**. Selain itu, tombol yang mengubah status karakter seperti **"Berdiri"**, **"Tengkurap"**, **"Terbang"**, dan **"Jongkok"** akan mencetak status yang sesuai dan memperbarui atribut state. Jika input tidak dikenali, program mencetak **"Perintah tidak dikenali"**. Kode ini berguna

untuk mensimulasikan perubahan posisi karakter berdasarkan input pengguna dalam sebuah game.

4. Membuat kelas Program.cs

```
JM4 | Program | Main(string[] args)

using System;

0 references
class Program
{
    0 references
    static void Main(string[] args)
    {
        // Implementasi Table-Driven
        Console.WriteLine("Kode Buah Apel: " + KodeBuah.GetKodeBuah("Apel"));
        Console.WriteLine("Kode Buah Durian: " + KodeBuah.GetKodeBuah("Durian"));

        // Implementasi State-Based Construction
        PosisiKarakterGame karakter = new PosisiKarakterGame();
        karakter.TekanTombol("W");
        karakter.TekanTombol("S");
        karakter.TekanTombol("Terbang");
        karakter.TekanTombol("Jongkok");
    }
}
```

Kode di atas adalah program dalam **C#** yang mengimplementasikan dua konsep utama: **Table-Driven** dan **State-Based Construction**. Pada bagian pertama, program menggunakan metode KodeBuah.GetKodeBuah("Apel") dan KodeBuah.GetKodeBuah("Durian") untuk mencetak kode buah berdasarkan nama, yang mengacu pada **Table-Driven** approach dengan **Dictionary** dari kelas KodeBuah. Selanjutnya, program membuat objek PosisiKarakterGame untuk mengelola perubahan posisi karakter dalam game. Dengan memanggil metode TekanTombol(), program mencetak status berdasarkan input tombol seperti "W" untuk arah atas, "S" untuk arah bawah, "Terbang" untuk posisi take-off, dan "Jongkok" untuk landing, mengikuti **State-Based Construction**. Kode ini menunjukkan bagaimana data dapat dikendalikan dengan tabel pencarian serta bagaimana perubahan kondisi karakter dapat ditangani secara dinamis dalam game.

5. Hasil Output

```
Microsoft Visual Studio Debu

Kode Buah Apel: A00
Kode Buah Durian: L00
Tombol arah atas ditekan
Tombol arah bawah ditekan
Posisi take off
Posisi landing

D:\KPL_FarhanKurniawan_2311104073_SE-07-02\KPL_FarhanKurniawan_2311104073\04_Automata_dan_Table-Driven_Construction\JM 4
\JM4\JM4\bin\Debug\net8.0\JM4.exe (process 24188) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .
```